



LESSON 08: LOOPS CONTINUED

Advanced Loop Control & Patterns

Module: Control Flow Mastery

Neon City Cyber Academy

Junior Agent Training Protocol

Your Team Today

Emily Chen - Handler

"Now that you know the basics, it's time to master the advanced loop techniques the pros use every day."

Cole Martinez - Tech Lead

"break, continue, and advanced range() - these are your power tools. Let me show you how to wield them."



Quick Review: Lesson 07

Last time we learned:

-  `while` loops - repeat while condition is true
-  `for` loops - iterate a specific number of times
-  `range()` - generate number sequences
-  Infinite loops - and how to avoid them

Today: Level up with advanced control!

Today's Advanced Arsenal

1. `break` - Emergency exit from loops
2. `continue` - Skip to next iteration
3. Advanced `range()` - Custom sequences
4. Loop patterns - Real-world techniques
5. Optimization - Write efficient loops

The `break` Statement

Purpose: Exit a loop immediately, no matter what.

```
for i in range(100):  
    if i == 5:  
        break # Stop the loop right now!  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Loop stops when `i` becomes 5!

Example: Search Mission

Task: Find a target in a data stream.

```
target = 42
found = False

for num in range(1, 100):
    if num == target:
        print(f"🎯 Target {target} found!")
        found = True
        break
    print(f"Scanning... {num}")

if not found:
    print("Target not found")
```

Why **break** ? No need to check remaining numbers once we find it!



Emily's Wisdom: When to Use `break`

"Use `break` when:

1. You're `searching` for something specific
2. An `error condition` occurs mid-loop
3. The user wants to `exit early`

Think of it as your emergency eject button. Once you find what you need or hit a problem, there's no reason to keep looping!"

The `continue` Statement

Purpose: Skip the rest of this iteration, jump to the next one.

```
for i in range(10):  
    if i % 2 == 0:  
        continue # Skip even numbers  
    print(i)
```

Output:

```
1  
3  
5  
7  
9
```

Example: Data Filtering

Task: Process valid records, skip corrupted ones.

```
records = [100, -1, 200, -1, 300, 400]

for record in records:
    if record == -1:
        print("⚠ Corrupted record, skipping...")
        continue

    # Process valid record
    print(f"✅ Processing: {record}")
```

Cleaner than nested if statements!



Cole's Pro Tip: `break` vs `continue`

`break` = I'm done with this entire loop

`continue` = I'm done with THIS iteration

Think of it like:

- `break` = Close the entire file
- `continue` = Skip to the next line

Both are powerful, but they do very different things!"

¹²₃₄ Advanced `range()`: Custom Steps

Remember: `range(start, stop, step)`

```
# Count by 2s
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8

# Count by 5s
for i in range(0, 51, 5):
    print(i)
# Output: 0, 5, 10, 15, ..., 50
```



Counting Backwards

Negative steps!

```
for i in range(10, 0, -1):  
    print(i)  
print("🚀 Liftoff!")
```

Output:

```
10  
9  
8  
...  
1  
🚀 Liftoff!
```

Note: `range(10, 0, -1)` goes from 10 down to 1 (stops before 0)



Challenge #1: Even Numbers Only

Mission: Print all even numbers from 0 to 20.

Hint: Use `range()` with a custom step!

```
# Your code here
```



Solution: Even Numbers

Method 1: Smart range()

```
for i in range(0, 21, 2):  
    print(i)
```

Method 2: Using modulo

```
for i in range(21):  
    if i % 2 == 0:  
        print(i)
```

Which is better? Method 1 - fewer iterations!

Emily's Efficiency Insight

"Always think: can I make the loop run fewer times?

Bad: Loop 100 times, skip 50

Good: Loop only the 50 you need

That's `range(0, 100, 2)` instead of `range(100)` with `if i % 2 == 0`.

Your CPU will thank you!"

🎮 Loop Pattern: Accumulator

Common pattern: Build up a result over iterations.

```
total = 0
for i in range(1, 11):
    total = total + i
print(f"Sum of 1-10: {total}")
```

Output: Sum of 1-10: 55

Pattern:

1. Initialize accumulator (`total = 0`)
2. Update it each iteration (`total = total + i`)

🎮 Loop Pattern: Counter

Count how many items meet a condition.

```
count = 0
for i in range(1, 101):
    if i % 7 == 0: # Divisible by 7?
        count = count + 1

print(f"Numbers divisible by 7: {count}")
```

Output: Numbers divisible by 7: 14

Loop Pattern: Flag Variable

Track if something was found.

```
password = "cyber2026"
authenticated = False

for attempt in range(3):
    guess = input("Password: ")
    if guess == password:
        authenticated = True
        break
    print(f"Wrong! {2 - attempt} attempts left.")

if authenticated:
    print("✅ Access granted")
else:
    print("❌ Locked out")
```

Cole's Pattern Library

"Every experienced programmer has a mental library of patterns:

 **Accumulator** - Sum, product, concatenation

 **Counter** - How many match a condition

 **Flag** - Did something happen

 **Search** - Find specific item

 **Min/Max** - Track extremes

Master these 5 patterns, and you can solve 80% of loop problems!"

Challenge #2: Find Maximum

Mission: Find the largest number in range 1-100.

Wait... that's easy, it's 100! 😊

Real task: Find the largest **random** number in 10 rolls.

```
import random

max_value = 0
# Your loop here
print(f"Max value: {max_value}")
```



Solution: Find Maximum

```
import random

max_value = 0
for i in range(10):
    num = random.randint(1, 100)
    print(f"Roll {i+1}: {num}")

    if num > max_value:
        max_value = num

print(f"🏆 Max value: {max_value}")
```

Pattern: Track the biggest seen so far!



Nested Loops Preview

Loops inside loops!

```
for i in range(3):  
    for j in range(3):  
        print(f"({i}, {j})")
```

Output:

```
(0, 0)  
(0, 1)  
(0, 2)  
(1, 0)  
...  
(2, 2)
```

We'll deep-dive into nested loops in Lesson 09!



Emily's Warning About Performance

"Nested loops multiply iterations:

- 1 loop of 100 = 100 iterations 
- 2 loops of 100 = 10,000 iterations 
- 3 loops of 100 = 1,000,000 iterations 

Each level you nest, you multiply the work. Use them when necessary, but be aware of the cost!"

Challenge #3: FizzBuzz

Classic Interview Question!

Print numbers 1-30, but:

- If divisible by 3 → print "Fizz"
- If divisible by 5 → print "Buzz"
- If divisible by both → print "FizzBuzz"
- Otherwise → print the number

```
# Your code here
```



Solution: FizzBuzz

```
for i in range(1, 31):  
    if i % 15 == 0: # Divisible by both 3 and 5  
        print("FizzBuzz")  
    elif i % 3 == 0:  
        print("Fizz")  
    elif i % 5 == 0:  
        print("Buzz")  
    else:  
        print(i)
```

Key: Check `% 15` first (most specific condition)!



Cole's Interview Insight

"FizzBuzz is a famous programming interview question. Why?

It tests:

1. Can you use loops?
2. Can you use conditionals?
3. Can you handle **multiple conditions**?
4. Do you check conditions in the **right order**?

Many candidates fail step 4 - they check **% 3** first and never reach the **% 15** case!"



Common Error: Wrong Range Bounds

Problem: Off-by-one errors

```
# Wrong: Prints 0-9 (10 numbers)
for i in range(10):
    print(i)

# Right: Prints 1-10 (10 numbers)
for i in range(1, 11):
    print(i)
```

Remember: `range(10)` = 0 through 9, NOT 1 through 10!



Common Error: Forgetting to Update

Problem: Infinite loop in while

```
count = 0
while count < 5:
    print(count)
    # Forgot: count += 1
```

Fix: Always update your loop variable!

```
count = 0
while count < 5:
    print(count)
    count += 1 # ✓
```



Common Error: Using `break` Outside Loops

```
if x == 5:  
    break # ✗ SyntaxError!
```

Error: `break` only works **inside** loops!

Fix: Only use `break` and `continue` inside `for` or `while` loops.



Emily's Debugging Checklist for Loops

"When your loop misbehaves:

1. ☒ Print the loop variable each iteration
2. ☒ Check your range bounds (start, stop, step)
3. ☒ Ensure loop variables are updating
4. ☒ Look for infinite loop conditions
5. ☒ Verify indentation (Python is picky!)
6. ☒ Check if `break` / `continue` are in the right place"

Final Challenge: Prime Number Checker

Mission: Check if a number is prime.

Definition: Prime = only divisible by 1 and itself

```
num = int(input("Enter a number: "))
is_prime = True

# Your loop here (hint: check divisors from 2 to num-1)

if is_prime:
    print(f"{num} is prime! 🎯")
else:
    print(f"{num} is not prime.")
```



Solution: Prime Checker

```
num = int(input("Enter a number: "))
is_prime = True

if num < 2:
    is_prime = False
else:
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break # Found a divisor, no need to continue

if is_prime:
    print(f"{num} is prime! 🎯")
else:
    print(f"{num} is not prime.")
```

Optimization: `break` as soon as we find a divisor!

Key Takeaways

- ✓ `break` exits the entire loop immediately
- ✓ `continue` skips to the next iteration
- ✓ `range(start, stop, step)` creates custom sequences
- ✓ **Loop patterns** solve common problems (accumulator, counter, flag)
- ✓ **Efficiency matters** - fewer iterations = faster code
- ✓ Always test **edge cases** and **boundary conditions**

Next Mission: Lesson 09

Coming Up:

- Nested loops (loops within loops!)
- 2D grids and matrices
- Time complexity ($O(n^2)$)
- When to use (and avoid) nested structures

Agent Status: Advanced Loop Control MASTERED 



Mission Stats

Concept: Advanced Loops (break, continue, patterns)

Difficulty: ★★★★★

Skills Unlocked:

- Loop control flow
- Optimization techniques
- Pattern recognition
- Real-world problem solving

Next Level: Nested Control Structures

